软件分析与安全 第一次实验报告

lab1

准备原始C代码：

```
#include <stdio.h>
int main() {
    int a = 50, b = 60;
    int c = 20, d = 30;
    printf("a+b=%d, c+d=%d\n", a+b, c+d);
    return 0;
}
```

写入到test.c

```
meiyue@meiyue-pc:~/llvm_lab1$ cat > test.c << 'EOF'
> #include <stdio.h>
> int main() {
> int a = 50, b = 60;
> int c = 20, d = 30;
> printf("a+b=%d, c+d=%d\n", a+b, c+d);
> return 0;
> }
> EOF
```

编写插桩代码：

```
#include "llvm/IR/LLVMContext.h"
#include "llvm/IR/Module.h"
#include "llvm/IR/Function.h"
#include "llvm/IR/BasicBlock.h"
#include "llvm/IR/IRBuilder.h"
#include "llvm/IRReader/IRReader.h"
#include "llvm/Support/SourceMgr.h"
#include "llvm/Support/raw_ostream.h"
#include "llvm/IR/Instructions.h"
using namespace llvm;

Function* createNewAdd(Module &M, LLVMContext &Ctx) {
    Type *i32 = Type::getInt32Ty(Ctx);
    FunctionType *FT = FunctionType::get(i32, {i32, i32}, false);
    Function *F = Function::Create(FT, Function::ExternalLinkage, "NewAdd", &M);
    BasicBlock *entry = BasicBlock::Create(Ctx, "entry", F);
    BasicBlock *trueBB = BasicBlock::Create(Ctx, "true", F);
    BasicBlock *falseBB = BasicBlock::Create(Ctx, "false", F);
    BasicBlock *end = BasicBlock::Create(Ctx, "end", F);
    IRBuilder<> builder(entry);
    Value *a = F->getArg(0);
```

```
    Value *b = F->getArg(1);
    Value *sum = builder.CreateAdd(a, b, "sum");
    Value *cmp = builder.CreateICmpSGT(sum, ConstantInt::get(i32, 100), "cmp");
    builder.CreateCondBr(cmp, trueBB, falseBB);
    builder.SetInsertPoint(trueBB);
    Value *sub = builder.CreateSub(sum, ConstantInt::get(i32, 100), "sub");
    builder.CreateBr(end);
    builder.SetInsertPoint(falseBB);
    builder.CreateBr(end);
    builder.SetInsertPoint(end);
    PHINode *phi = builder.CreatePHI(i32, 2, "retval");
    phi->addIncoming(sub, trueBB);
    phi->addIncoming(sum, falseBB);
    builder.CreateRet(phi);
    return F;
}

void replaceAddWithCall(Module &M, Function *NewAdd) {
    for (auto &F : M) {
        if (F.isDeclaration() || &F == NewAdd) continue;
        for (auto &BB : F) {
            for (auto it = BB.begin(); it != BB.end(); ) {
                Instruction &I = *it++;
                if (I.getOpcode() == Instruction::Add && I.getType()-
>isIntegerTy(32)) {
                    IRBuilder<> builder(&I);
                    Value *call = builder.CreateCall(NewAdd, {I.getOperand(0),
I.getOperand(1)});
                    I.replaceAllUsesWith(call);
                    I.eraseFromParent();
                }
            }
        }
    }
}

int main(int argc, char** argv) {
    LLVMContext Ctx;
    SMDiagnostic Err;
    std::unique_ptr<Module> M = parseIRFile(argv[1], Err, Ctx);
    if (!M) return 1;
    Function *NewAdd = createNewAdd(*M, Ctx);
    replaceAddWithCall(*M, NewAdd);
    std::error_code EC;
    raw_fd_ostream out(argv[2], EC);
    M->print(out, nullptr);
    return 0;
}
```

```
meiyue@meiyue-pc:~/llvm_lab1$ cat > lab1.cpp << 'EOF'
>
> #include "llvm/IR/LLVMContext.h"
>
> #include "llvm/IR/Module.h"
>
```

编译 C++ 工具；

生成原始 IR 文件；

运行工具，生成最终 IR，最后查看结果：

```
meiyue@meiyue-pc:~/llvm_lab1$ g++ lab1.cpp -o lab1 `llvm-config --cxxflags --lib
s --system-libs`
meiyue@meiyue-pc:~/llvm_lab1$ clang -S -emit-llvm test.c -o test.ll
meiyue@meiyue-pc:~/llvm_lab1$ ./lab1 test.ll output.ll
meiyue@meiyue-pc:~/llvm_lab1$ cat output.ll
; ModuleID = 'test.ll'
source_filename = "test.c"
target datalayout = "e-m:e-p270:32:32-p271:32:32-p272:64:64-i64:64-f80:128-n8:16
:32:64-S128"
target triple = "x86_64-unknown-linux-gnu"

@.str = private unnamed_addr constant [16 x i8] c"a+b=%d, c+d=%d\0A\00", align 1

; Function Attrs: noinline nounwind optnone uwtable
define dso_local i32 @main() #0 {
```

插桩后的结果为output.ll：

```
; ModuleID = 'test.ll'
source_filename = "test.c"
target datalayout = "e-m:e-p270:32:32-p271:32:32-p272:64:64-i64:64-f80:128-
n8:16:32:64-S128"
target triple = "x86_64-unknown-linux-gnu"

@.str = private unnamed_addr constant [16 x i8] c"a+b=%d, c+d=%d\0A\00", align 1

; Function Attrs: noinline nounwind optnone uwtable
define dso_local i32 @main() #0 {
  %1 = alloca i32, align 4
  %2 = alloca i32, align 4
  %3 = alloca i32, align 4
  %4 = alloca i32, align 4
  %5 = alloca i32, align 4
  store i32 0, ptr %1, align 4
  store i32 50, ptr %2, align 4
  store i32 60, ptr %3, align 4
  store i32 20, ptr %4, align 4
```

```
  store i32 30, ptr %5, align 4
  %6 = load i32, ptr %2, align 4
  %7 = load i32, ptr %3, align 4
  %8 = call i32 @NewAdd(i32 %6, i32 %7)
  %9 = load i32, ptr %4, align 4
  %10 = load i32, ptr %5, align 4
  %11 = call i32 @NewAdd(i32 %9, i32 %10)
  %12 = call i32 (ptr, ...) @printf(ptr noundef @.str, i32 noundef %8, i32
noundef %11)
  ret i32 0
}

declare i32 @printf(ptr noundef, ...) #1

define i32 @NewAdd(i32 %0, i32 %1) {
entry:
  %sum = add i32 %0, %1
  %cmp = icmp sgt i32 %sum, 100
  br i1 %cmp, label %true, label %false

true:                                             ; preds = %entry
  %sub = sub i32 %sum, 100
  br label %end

false:                                            ; preds = %entry
  br label %end

end:                                              ; preds = %false, %true
  %retval = phi i32 [ %sub, %true ], [ %sum, %false ]
  ret i32 %retval
}

attributes #0 = { noinline nounwind optnone uwtable "frame-pointer"="all" "min-
legal-vector-width"="0" "no-trapping-math"="true" "stack-protector-buffer-
size"="8" "target-cpu"="x86-64" "target-
features"="+cx8,+fxsr,+mmx,+sse,+sse2,+x87" "tune-cpu"="generic" }
attributes #1 = { "frame-pointer"="all" "no-trapping-math"="true" "stack-
protector-buffer-size"="8" "target-cpu"="x86-64" "target-
features"="+cx8,+fxsr,+mmx,+sse,+sse2,+x87" "tune-cpu"="generic" }

!llvm.module.flags = !{!0, !1, !2, !3, !4}
!llvm.ident = !{!5}

!0 = !{i32 1, !"wchar_size", i32 4}
!1 = !{i32 7, !"PIC Level", i32 2}
!2 = !{i32 7, !"PIE Level", i32 2}
!3 = !{i32 7, !"uwtable", i32 2}
!4 = !{i32 7, !"frame-pointer", i32 2}
!5 = !{!"clang version 15.0.7"}
```

查看插桩前的原始IR文件:

```
; ModuleID = 'test.c'
source_filename = "test.c"
```

```
target datalayout = "e-m:e-p270:32:32-p271:32:32-p272:64:64-i64:64-f80:128-
n8:16:32:64-S128"
target triple = "x86_64-unknown-linux-gnu"

@.str = private unnamed_addr constant [16 x i8] c"a+b=%d, c+d=%d\0A\00", align 1

; Function Attrs: noinline nounwind optnone uwtable
define dso_local i32 @main() #0 {
  %1 = alloca i32, align 4
  %2 = alloca i32, align 4
  %3 = alloca i32, align 4
  %4 = alloca i32, align 4
  %5 = alloca i32, align 4
  store i32 0, ptr %1, align 4
  store i32 50, ptr %2, align 4
  store i32 60, ptr %3, align 4
  store i32 20, ptr %4, align 4
  store i32 30, ptr %5, align 4
  %6 = load i32, ptr %2, align 4
  %7 = load i32, ptr %3, align 4
  %8 = add nsw i32 %6, %7
  %9 = load i32, ptr %4, align 4
  %10 = load i32, ptr %5, align 4
  %11 = add nsw i32 %9, %10
  %12 = call i32 (ptr, ...) @printf(ptr noundef @.str, i32 noundef %8, i32
noundef %11)
  ret i32 0
}

declare i32 @printf(ptr noundef, ...) #1

attributes #0 = { noinline nounwind optnone uwtable "frame-pointer"="all" "min-
legal-vector-width"="0" "no-trapping-math"="true" "stack-protector-buffer-
size"="8" "target-cpu"="x86-64" "target-
features"="+cx8,+fxsr,+mmx,+sse,+sse2,+x87" "tune-cpu"="generic" }
attributes #1 = { "frame-pointer"="all" "no-trapping-math"="true" "stack-
protector-buffer-size"="8" "target-cpu"="x86-64" "target-
features"="+cx8,+fxsr,+mmx,+sse,+sse2,+x87" "tune-cpu"="generic" }

!llvm.module.flags = !{!0, !1, !2, !3, !4}
!llvm.ident = !{!5}

!0 = !{i32 1, !"wchar_size", i32 4}
!1 = !{i32 7, !"PIC Level", i32 2}
!2 = !{i32 7, !"PIE Level", i32 2}
!3 = !{i32 7, !"uwtable", i32 2}
!4 = !{i32 7, !"frame-pointer", i32 2}
!5 = !{!"clang version 15.0.7"}
```

和插桩后的output.ll对比，可验证插桩成功

编译运行替换后的 .ll 文件，验证最终答案，符合预期结果

```
meiyue@meiyue-pc:~/llvm_lab1$ clang output.ll -o test
meiyue@meiyue-pc:~/llvm_lab1$ ./test
a+b=10, c+d=50
meiyue@meiyue-pc:~/llvm_lab1$
```

lab2

编写编译程序 mycc

```
#include <string>
#include <cstdlib>
using namespace std;

int main(int argc, char *argv[])
{
    if (argc != 4 || string(argv[2]) != "-o")
        return 1;

    string src = argv[1];
    string out = argv[3];
    string tmp1 = "tmp.ll";
    string tmp2 = "inst.ll";

    string cmd1 = "clang -S -emit-llvm -O0 -Xclang -disable-O0-optnone " + src +
" -o " + tmp1;
    system(cmd1.c_str());

    string cmd2 = "./lab1 " + tmp1 + " " + tmp2;
    system(cmd2.c_str());

    string cmd3 = "clang " + tmp2 + " -o " + out;
    system(cmd3.c_str());

    string cmd4 = "rm -f " + tmp1 + " " + tmp2;
    system(cmd4.c_str());

    return 0;
}
EOF
```

```
meiyue@meiyue-pc:~/llvm_lab1$ cat > mycc.cpp << 'EOF'

>
> #include <string>
>
> #include <cstdlib>
>
> using namespace std;
>
>
```

编译 mycc，运行，验证结果

```
meiyue@meiyue-pc:~/llvm_lab1$ g++ mycc.cpp -o mycc
meiyue@meiyue-pc:~/llvm_lab1$ ./mycc test.c -o test_exec
meiyue@meiyue-pc:~/llvm_lab1$ ./test_exec
```

输出为预期值，实现了自减100操作，验证成功

```
meiyue@meiyue-pc:~/llvm_lab1$ ./test_exec
a+b=10, c+d=50
meiyue@meiyue-pc:~/llvm_lab1$
```